MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Gendal Lal

Subject & Subject Code: Theory of Computation & 4CSE4-03, 4AM4-03, 4AD4-03

Branch: CSE, AIML and AI & DS Batch:

Semester: IV **Session:** 2023-24(Even Sem.)

Unit No.: 4 **Date of Submission: 11/03/2024**

Lect.No.	Topics Covered	Page No.	Total pages
1	Turning machine basics	1-6	6
2	Turning machine as Adder	7-9	3
3	Turning Machine Examples	10-25	16
4	Universal Turning Machine	26-27	2
5	Multi-Track turning Machines	28-28	1
6	Properties of RL & REL Languages	29-31	3

References Book:

1. "Introduction to the Theory of Computation "by Micheal Sipser
2. "Introduction to computability "by Fred C Henie

FACULTY **HOD** **DEAN ACADEMICS**

UNIT-4 (TOC)

Turing Machine :Basics

Turing machine was invented in 1936 by **Alan Turing**. It is an accepting device which accepts Recursive Enumerable Language generated by type 0 grammar.

There are various features of the Turing machine:

1.It has an external memory which remembers arbitrary long sequence of input.

2.It has unlimited memory capability.

3.The model has a facility by which the input at left or right on the tape can be read easily.

4.The machine can produce a certain output based on its input. Sometimes it may be required that the same input has to be used to generate the output. So in this machine, the distinction between input and output has been removed. Thus a common set of alphabets can be used for the Turing machine.

Formal definition of Turing machine:

A Turing machine can be defined as a collection of 7 components:

Q: the finite set of states

∑: the finite set of input symbols

T: the tape symbol

q0: the initial state

MTRC

F: a set of final states
B: a blank symbol used as a end marker for input
δ: a transition or mapping function.

The mapping function shows the mapping from states of finite automata and input symbol on the tape to the next states, external symbols and the direction for moving the tape head. This is known as a triple or a program for turing machine.

(q0, a) → (q1, A,
R)

That means in q0 state, if we read symbol 'a' then it will go to state q1, replaced a by X and move ahead right(R stands for right).

Example:

Construct TM for the language $L = \{0^n1^n\}$ where n>=1.

Solution:

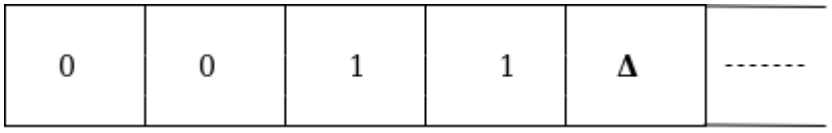
We have already solved this problem by PDA. In PDA, we have a stack to remember the previous symbol. The main advantage of the Turing machine is we have a tape head which can be moved forward or backward, and the input tape can be scanned.

The simple logic which we will apply is read out each '0' mark it by A and then move ahead along with the input tape and find out
1 convert it to B. Now, repeat this process for all a's and b's.

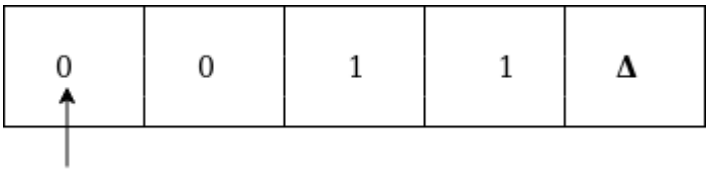
Now we will see how this turing machine work for 0011.

The simulation for 0011 can be shown as below:

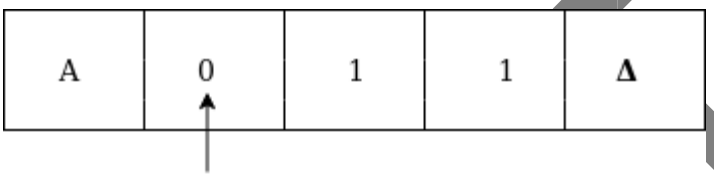
MITRC



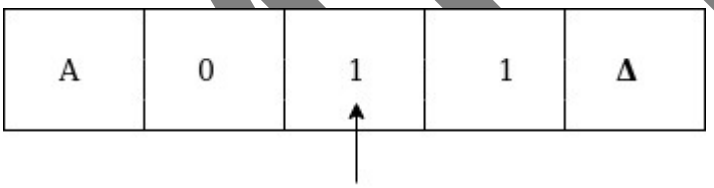
Now, we will see how this turing machine will works for 0011. Initially, state is q0 and head points to 0 as:



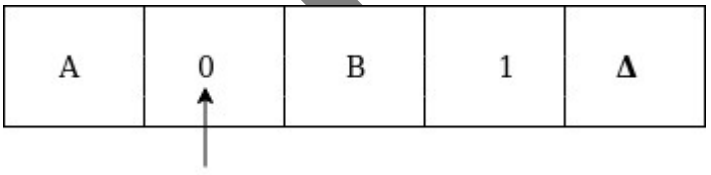
The move will be δ(q0, 0) = δ(q1, A, R) which means it will go to state q1, replaced 0 by A and head will move to the right as:



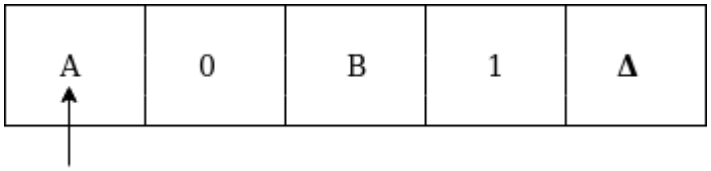
The move will be δ(q1, 0) = δ(q1, 0, R) which means it will not change any symbol, remain in the same state and move to the right as:



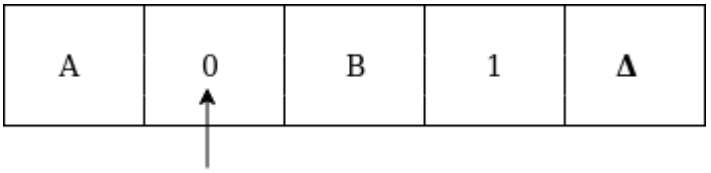
The move will be δ(q1, 1) = δ(q2, B, L) which means it will go to state q2, replaced 1 by B and head will move to left as:



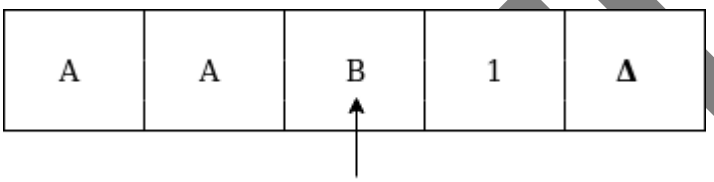
Now move will be δ(q2, 0) = δ(q2, 0, L) which means it will not change any symbol, remain in the same state and move to left as:



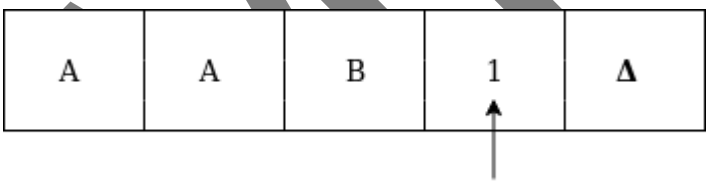
The move will be δ(q2, A) = δ(q0, A, R), it means will go to state q0, replaced A by A and head will move to the right as:



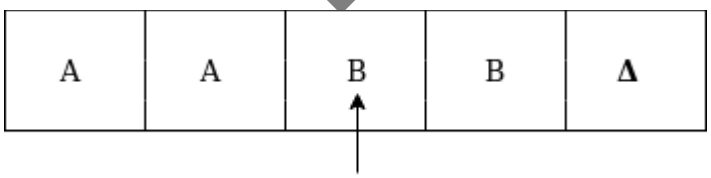
The move will be δ(q0, 0) = δ(q1, A, R) which means it will go to state q1, replaced 0 by A, and head will move to right as:



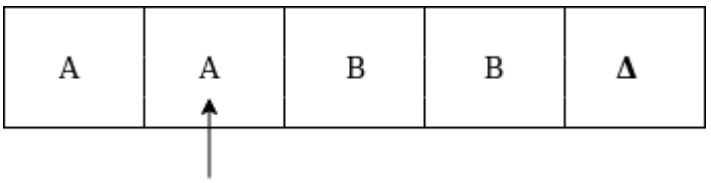
The move will be δ(q1, B) = δ(q1, B, R) which means it will not change any symbol, remain in the same state and move to right as:



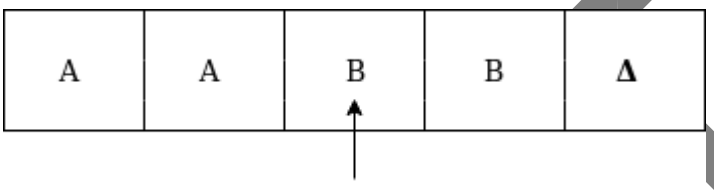
The move will be δ(q1, 1) = δ(q2, B, L) which means it will go to state q2, replaced 1 by B and head will move to left as:



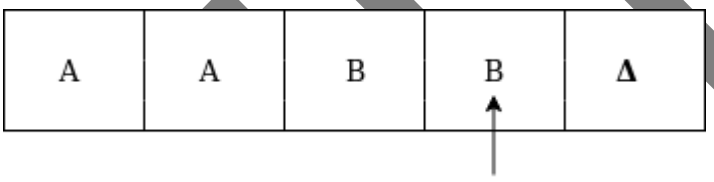
The move δ(q2, B) = (q2, B, L) which means it will not change any symbol, remain in the same state and move to left as:



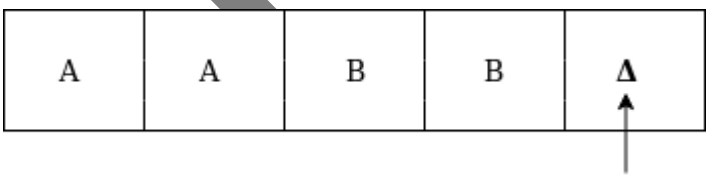
Now immediately before B is A that means all the 0?s are market by A. So we will move right to ensure that no 1 is present. The move will be δ(q2, A) = (q0, A, R) which means it will go to state q0, will not change any symbol, and move to right as:



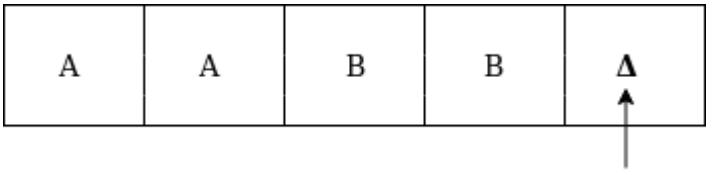
The move δ(q0, B) = (q3, B, R) which means it will go to state q3, will not change any symbol, and move to right as:



The move δ(q3, B) = (q3, B, R) which means it will not change any symbol, remain in the same state and move to right as:



The move δ(q3, Δ) = (q4, Δ, R) which means it will go to state q4 which is the HALT state and HALT state is always an accept state for any TM.



MITRC

LECTURE NO:2

Turing machine as Adder

Turing machine as Adder

Approach for Addition

5.Numbers are given in Uniary form

6.Example: 3 = 111, 2 = 11, 5 = 11111 etc.

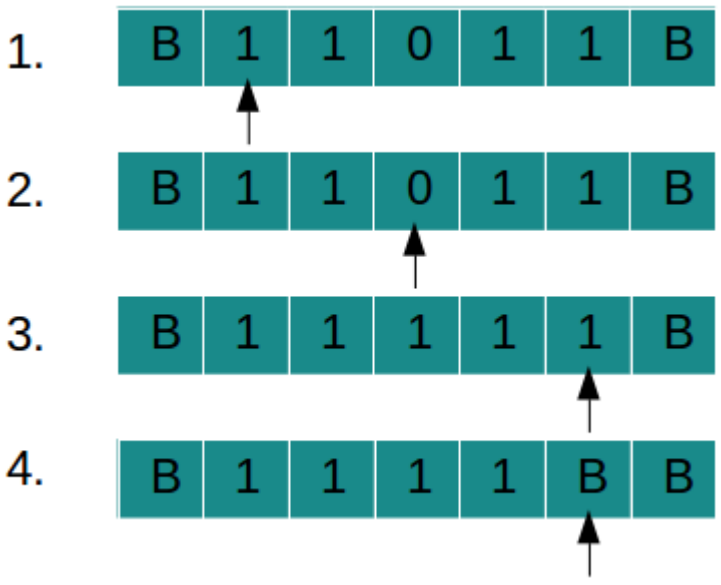
7.For addition of 3 and 4, numbers will be given in TAPE as "B B 1 1 1 0 1 1 1 1 B B".

8.Convert '0' to '1' and reduce '1' from right

9.Hence output will be "B B 1 1 1 1 1 1 1 B B B"

10. Total number of '1' in output is = 7 which is addition of 3 and 4

TAPE movement for string "110111":



MITRC

Explanation of TAPE movement

1. Input is given as "11011" (2 + 2)
2. Scan string from left to right
3. Pass all '1'S and convert '0' to '1'
4. Reach BLANK in right
5. Convert rightmost '1' to BLANK

Addition of 2 and 2 = 4 is written on TAPE
Input String : 11011
Output String : 1111

State Transition Diagram

We have designed state transition diagram for adder as follows:

Start scanning string from left to right

Pass all '1's and keep moving right

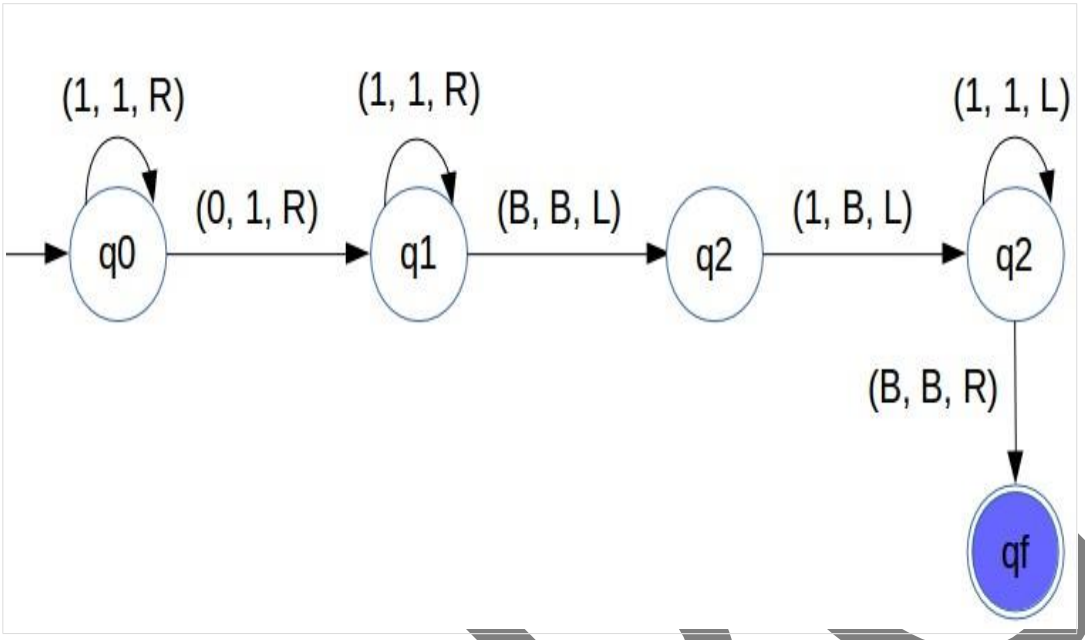
Convert '0' to '1' and keep moving right

When BLANK(in right) is reached move one step left convert '1' to BLANK

Keep moving left after that to point start of string

Number of '1's is reduced because number of '1' was increased by one while converting '0' to '1'

MITRC



Uniary addition is like string concatination.

MITRC

LECTURE NO:3

Turing Machine Examples

Example 1:

Construct a TM for the language $L = \{0^n1^n2^n\}$ where $n \geq 1$

Solution:

$L = \{0^n1^n2^n \mid n \geq 1\}$ represents language where we use only 3 character, i.e., 0, 1 and 2. In this, some number of 0's followed by an equal number of 1's and then followed by an equal number of 2's. Any type of string which falls in this category will be accepted by this language.

The simulation for 001122 can be shown as below:

0	0	1	1	2	2	X	-------

Now, we will see how this Turing machine will work for 001122. Initially, state is q0 and head points to 0 as:

0	0	1	1	2	2	X

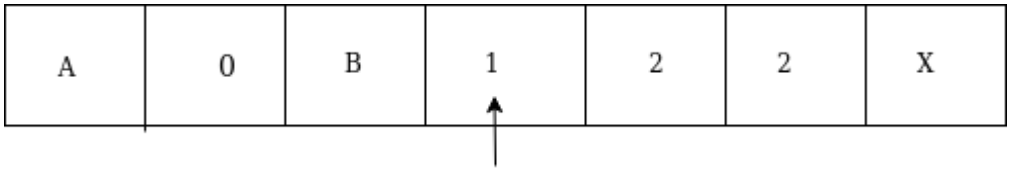
The move will be $\delta(q0, 0) = \delta(q1, A, R)$ which means it will go to state q1, replaced 0 by A and head will move to the right as:

A	0	1	1	2	2	X

The move will be $\delta(q1, 0) = \delta(q1, 0, R)$ which means it will not change any symbol, remain in the same state and move to the right as:

A	0	1	1	2	2	X

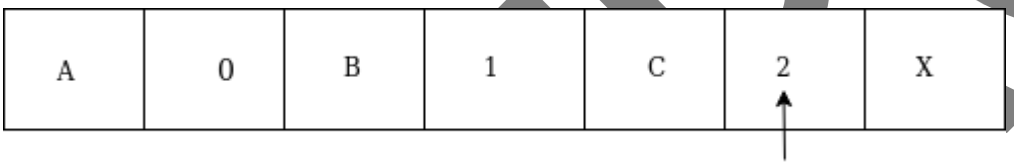
The move will be δ(q1, 1) = δ(q2, B, R) which means it will go to state q2, replaced 1 by B and head will move to right as:



The move will be δ(q2, 1) = δ(q2, 1, R) which means it will not change any symbol, remain in the same state and move to right as:



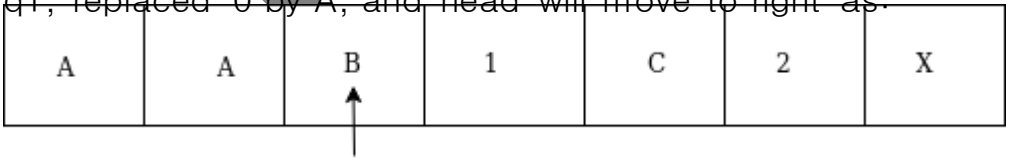
The move will be δ(q2, 2) = δ(q3, C, R) which means it will go to state q3, replaced 2 by C and head will move to right as:



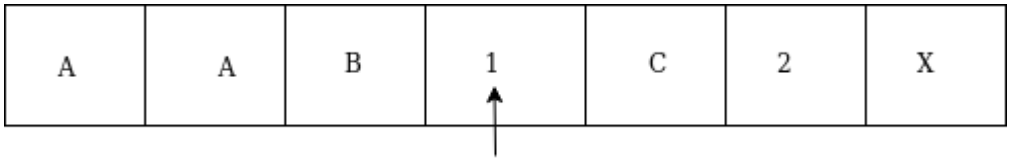
Now move δ(q3, 2) = δ(q3, 2, L) and δ(q3, C) = δ(q3, C, L) and δ (q3,
1) = δ(q3, 1, L) and δ(q3, B) = δ(q3, B, L) and δ(q3, 0) = δ(q3, 0, L), and then move δ(q3, A) = δ(q0, A, R), it means will go to state q0, replaced A by A and head will move to right as:



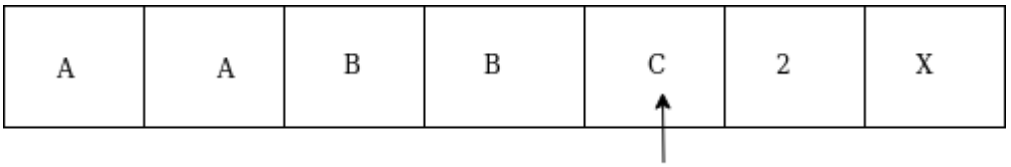
The move will be δ(q0, 0) = δ(q1, A, R) which means it will go to state q1, replaced 0 by A, and head will move to right as:



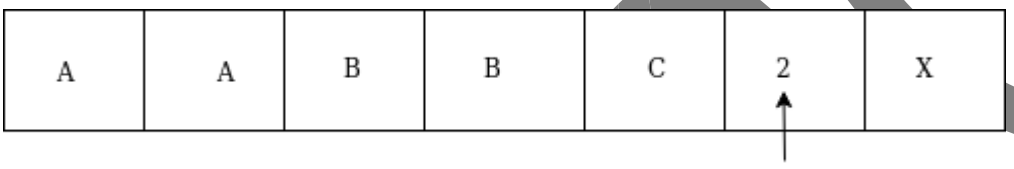
The move will be δ(q1, B) = δ(q1, B, R) which means it will not change any symbol, remain in the same state and move to right as:



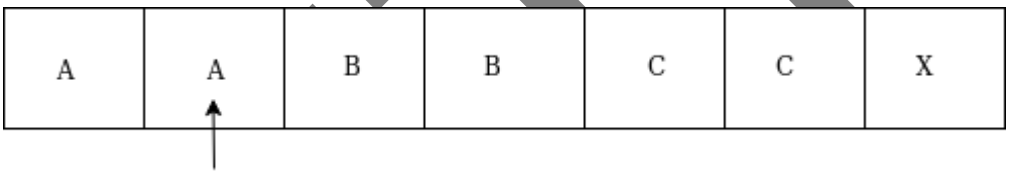
The move will be δ(q1, 1) = δ(q2, B, R) which means it will go to state q2, replaced 1 by B and head will move to right as:



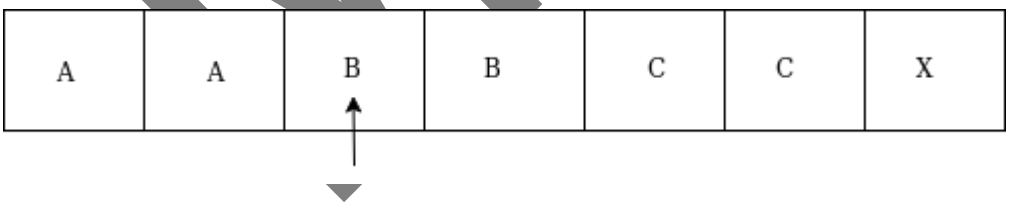
The move will be δ(q2, C) = δ(q2, C, R) which means it will not change any symbol, remain in the same state and move to right as:



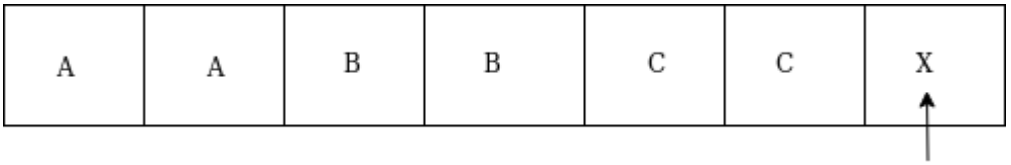
The move will be δ(q2, 2) = δ(q3, C, L) which means it will go to state q3, replaced 2 by C and head will move to left until we reached A as:



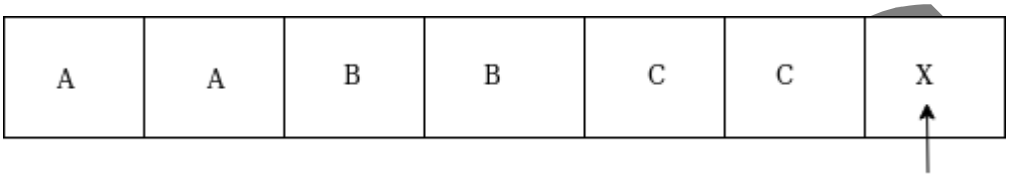
immediately before B is A that means all the 0's are market by A. So we will move right to ensure that no 1 or 2 is present. The move will be δ(q2, B) = (q4, B, R) which means it will go to state q4, will not change any symbol, and move to right as:



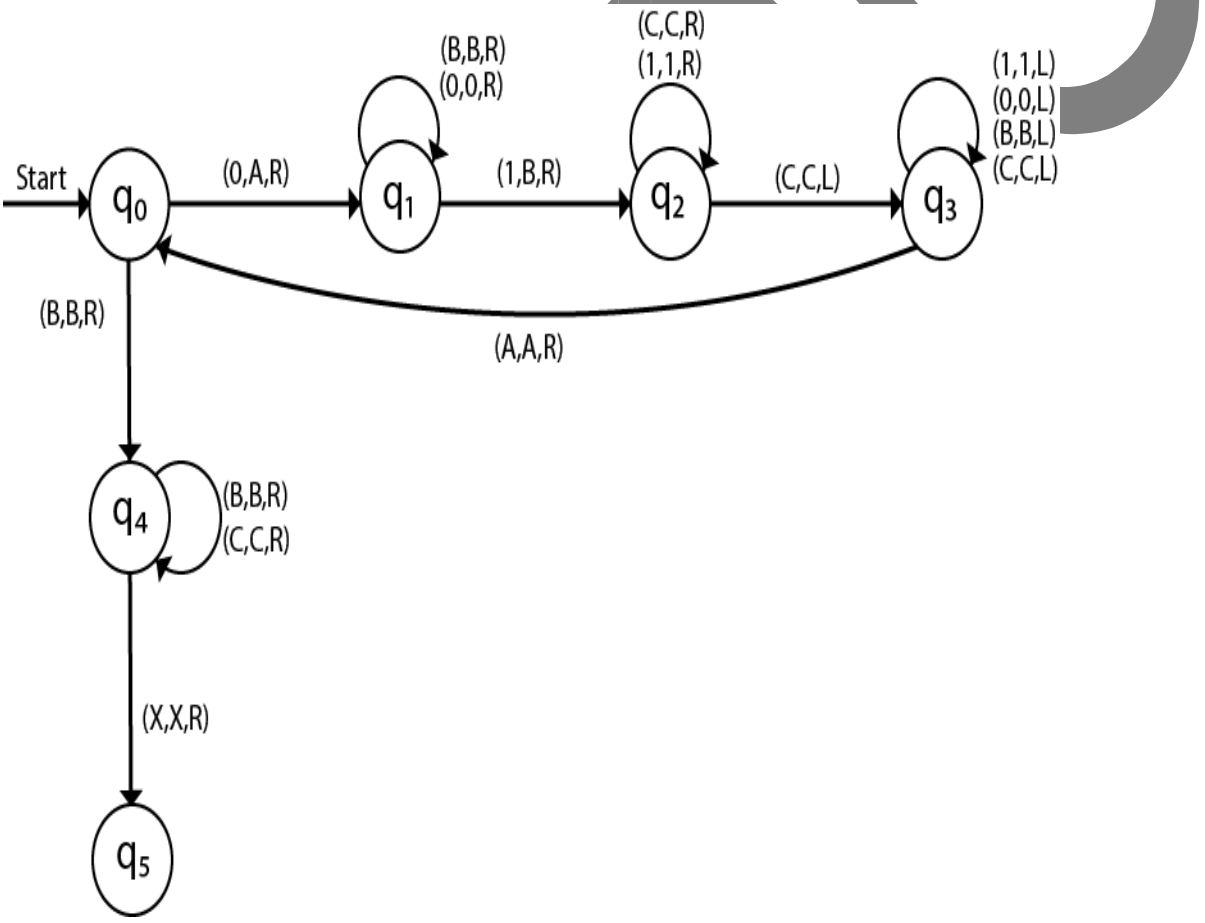
The move will be (q4, B) = δ(q4, B, R) and (q4, C) = δ(q4, C, R) which means it will not change any symbol, remain in the same state and move to right as:



The move δ(q4, X) = (q5, X, R) which means it will go to state q5 which is the HALT state and HALT state is always an accept state for any TM.



The same TM can be represented by Transition Diagram:



Example 2:

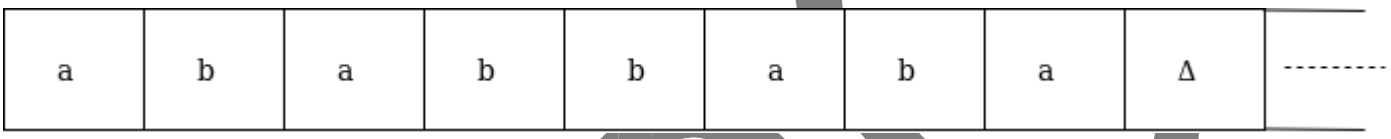
Construct a TM machine for checking the palindrome of the string of even length.

Solution:

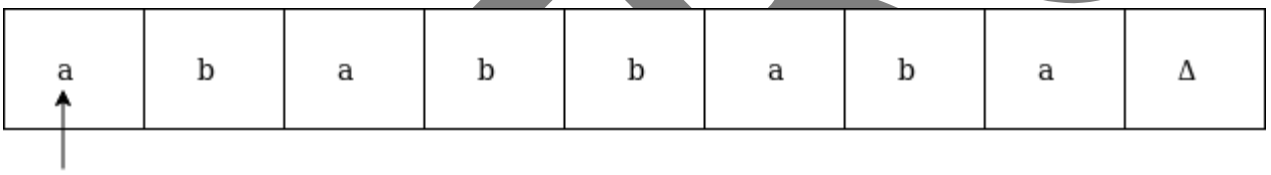
Firstly we read the first symbol from the left and then we compare it with the first symbol from right to check whether it is the same.

Again we compare the second symbol from left with the second symbol from right. We repeat this process for all the symbols. If we found any symbol not matching, we cannot lead the machine to HALT state.

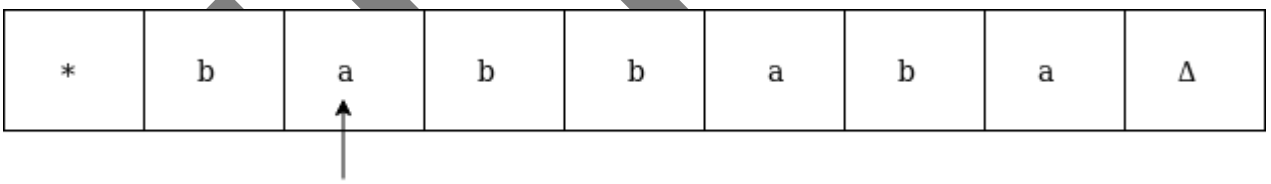
Suppose the string is ababbabaΔ. The simulation for ababbabaΔ can be shown as follows:



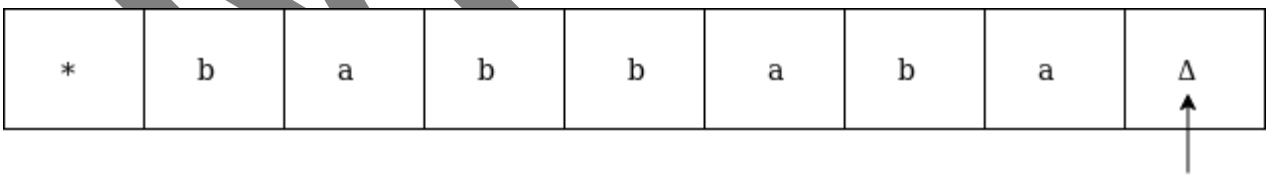
Now, we will see how this Turing machine will work for ababbabaΔ. Initially, state is q0 and head points to a as:



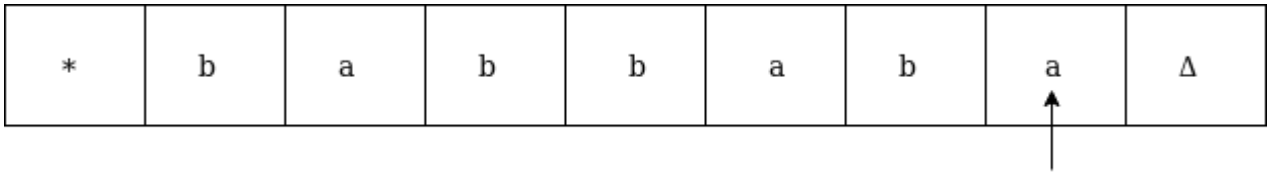
We will mark it by * and move to right end in search of a as:



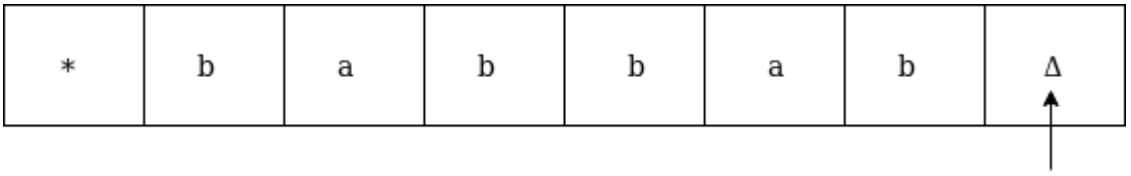
We will move right up to Δ as:



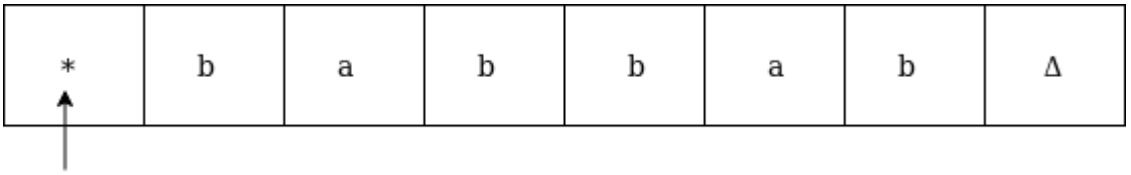
We will move left and check if it is a:



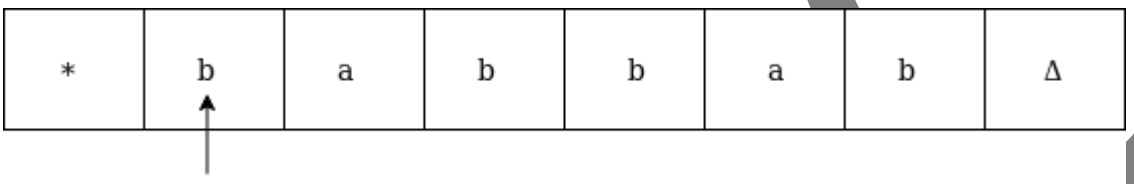
It is 'a' so replace it by Δ and move left as:



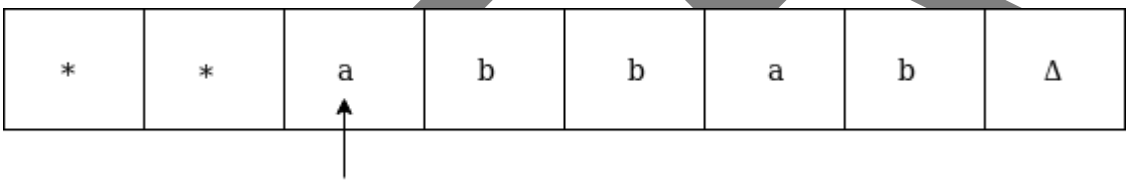
Now move to left up to * as:



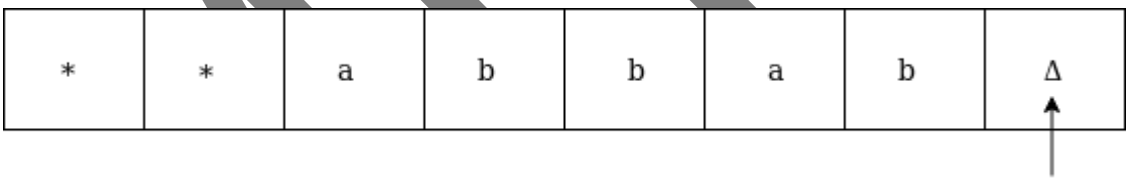
Move right and read it



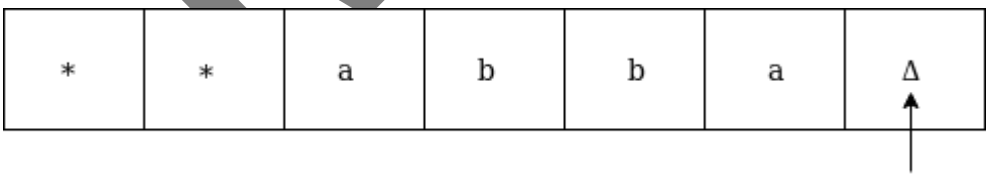
Now convert b by * and move right as:



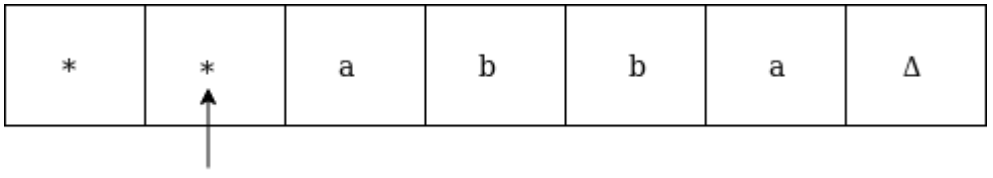
Move right up to Δ in search of b as:



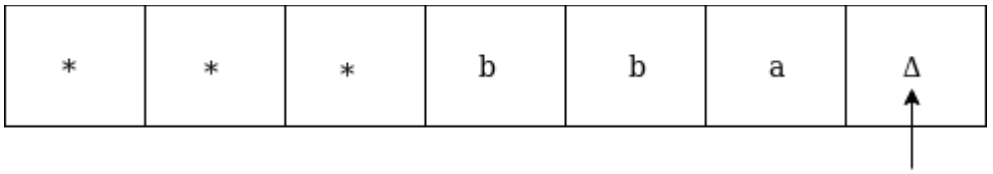
Move left, if the symbol is b then convert it into Δ as:



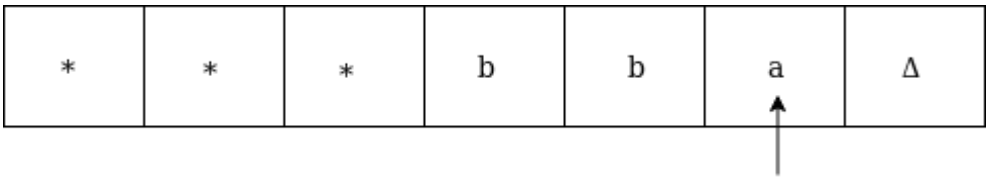
Now move left until * as:



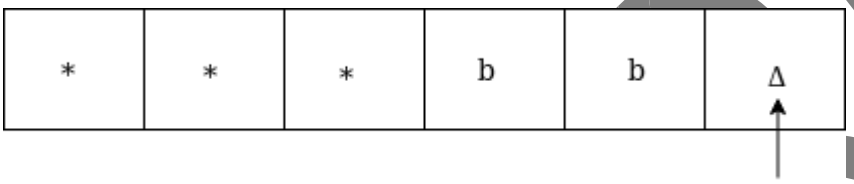
Replace a by * and move right up to Δ as:



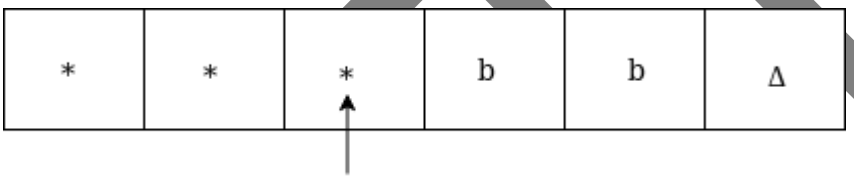
We will move left and check if it is a, then replace it by Δ as:



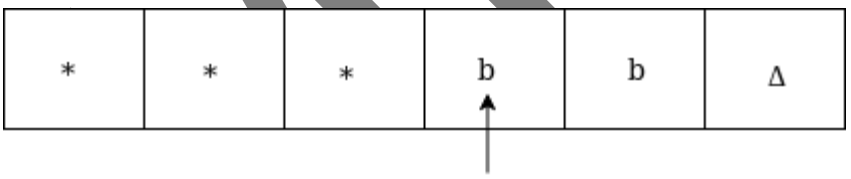
It is 'a' so replace it by Δ as:



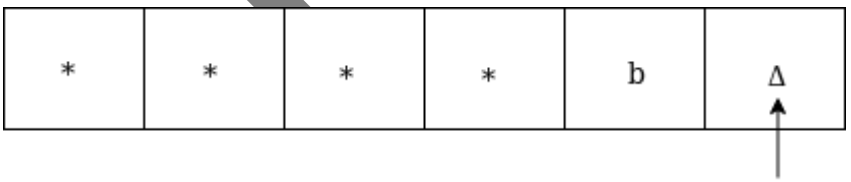
Now move left until *



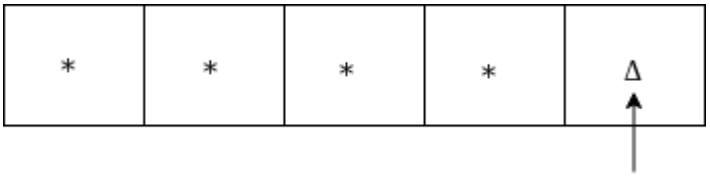
Now move right as:



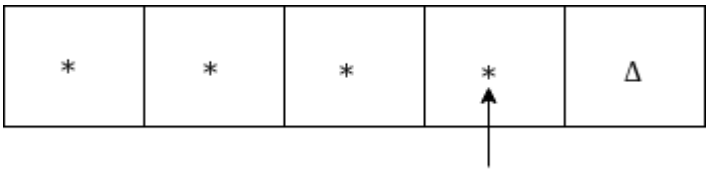
Replace b by * and move right up to Δ as:



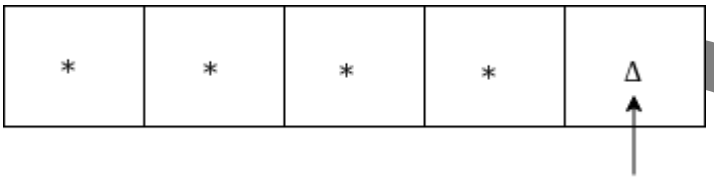
Move left, if the left symbol is b, replace it by Δ as:



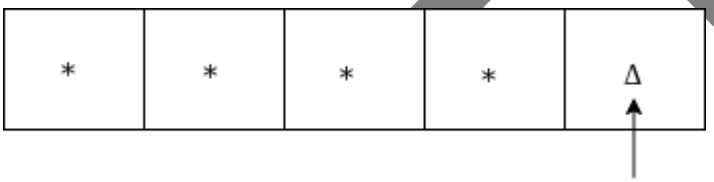
Move left till *



Move right and check whether it is Δ

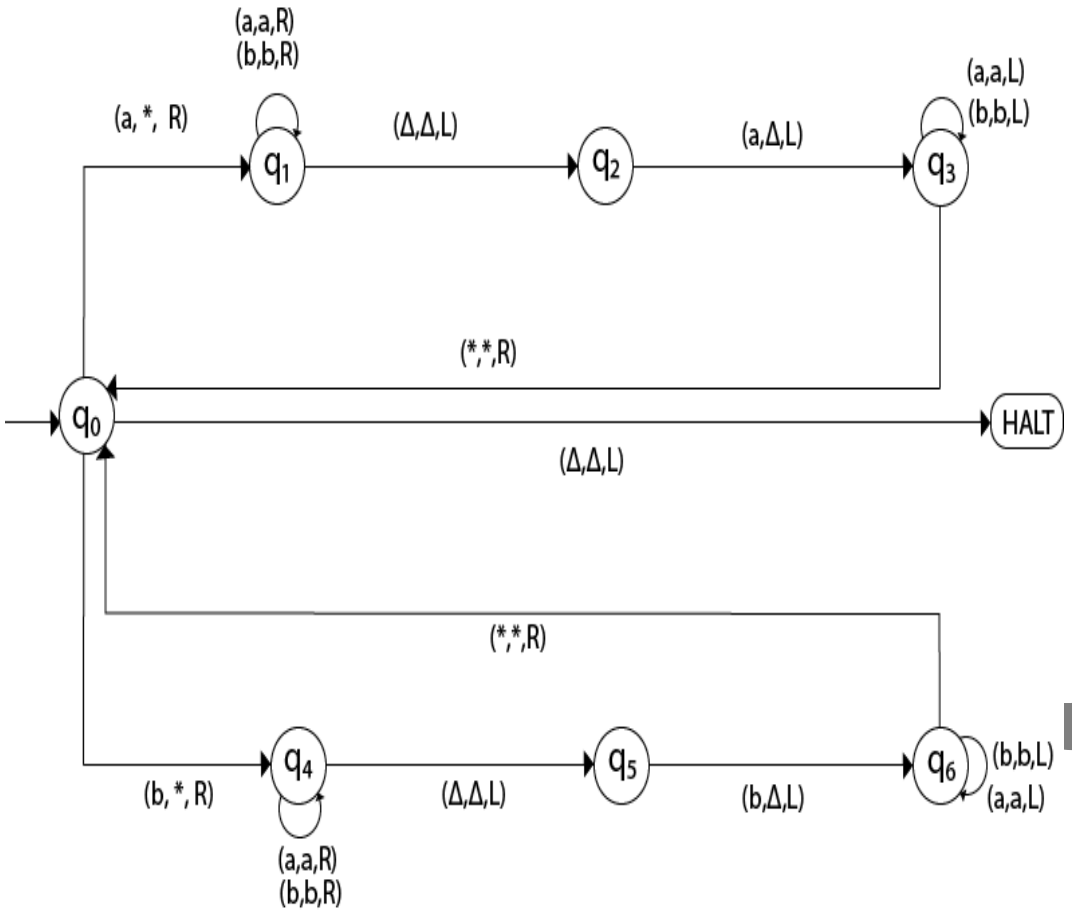


Go to HALT state



The same TM can be represented by Transition Diagram:

MIRC



Example 3:

Construct a TM machine for checking the palindrome of the string of odd length.

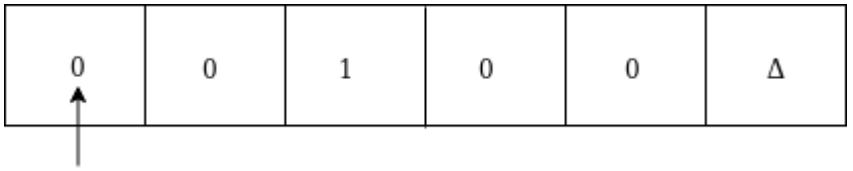
Solution:

Firstly we read the first symbol from left and then we compare it with the first symbol from right to check whether it is the same.

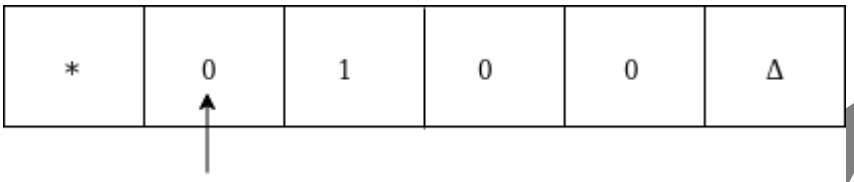
Again we compare the second symbol from left with the second symbol from right. We repeat this process for all the symbols. If we found any symbol not matching, we lead the machine to HALT state.

Suppose the string is 00100Δ. The simulation for 00100Δ can be shown as follows:

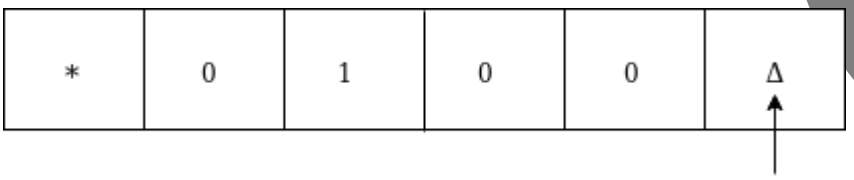
Now, we will see how this Turing machine will work for 00100Δ. Initially, state is q0 and head points to 0 as:



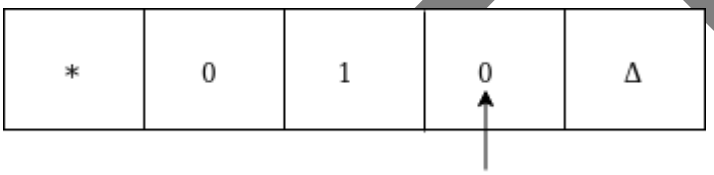
Now replace 0 by * and move right as:



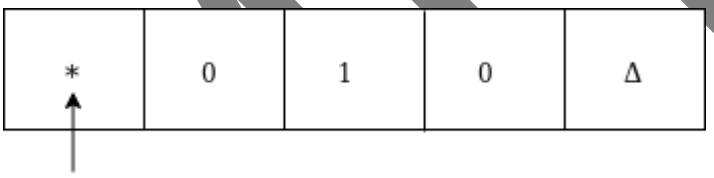
Move right up to Δ as:



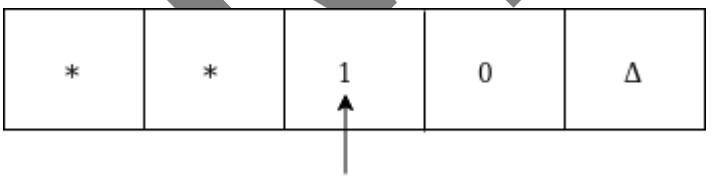
Move left and replace 0 by Δ and move left:



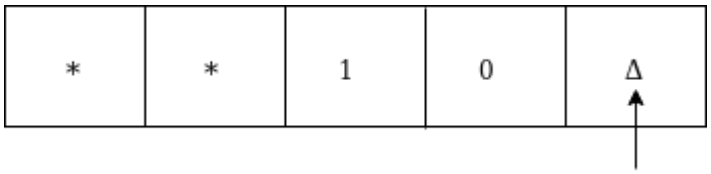
Now move left up to * as:



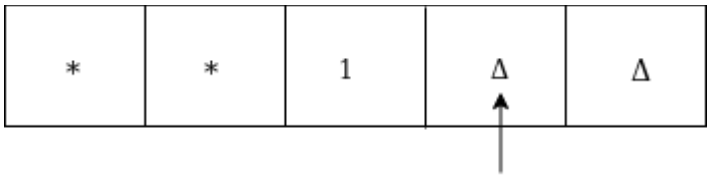
Move right, convert 0 by * and then move right as:



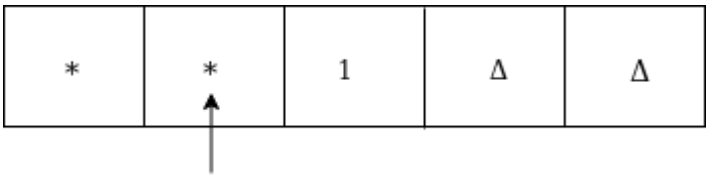
Moved right up to Δ



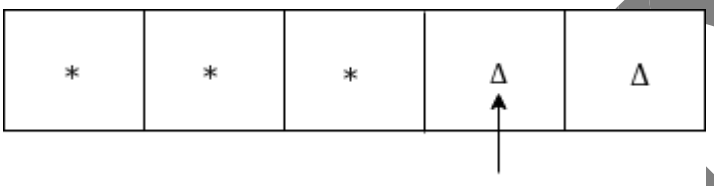
Move left and replace 0 by Δ as:



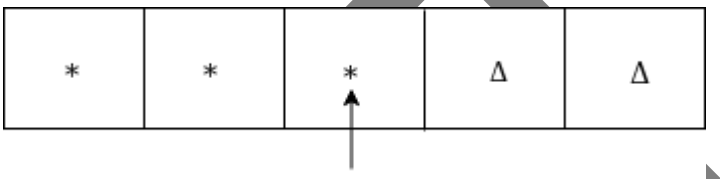
Move left till * as:



Move right and convert 1 to * as:



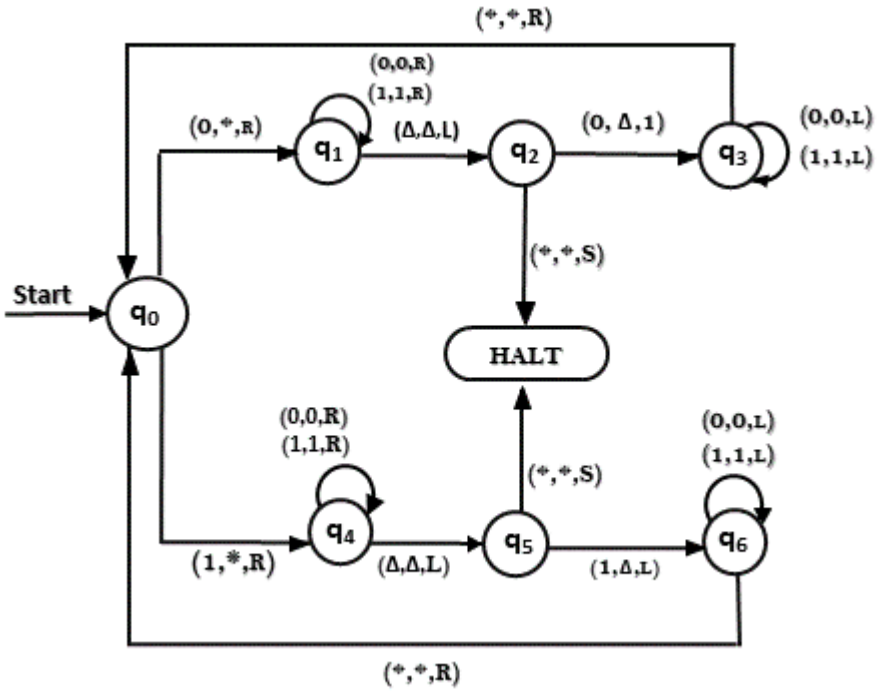
Move left



Since it is *, goto HALT state.

The same TM can be represented by Transition Diagram:

M.RC



Example 4:

Construct TM for the addition function for the unary number system.

Solution:

The unary number is made up of only one character, i.e. The number 5 can be written in unary number system as 11111. In this TM, we are going to perform the addition of two unary numbers.

For example

2 + 3

i.e. 11 + 111 = 11111

If you observe this process of addition, you will find the resemblance with string concatenation function.

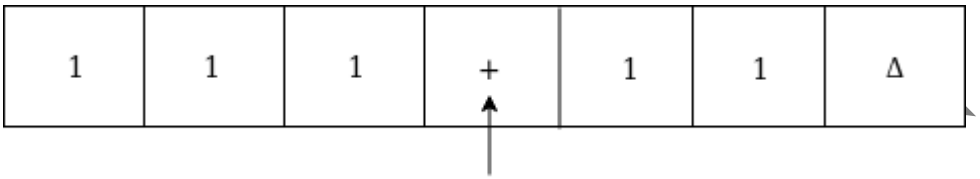
In this case, we simply replace + by 1 and move ahead right for searching end of the string we will convert last 1 to Δ.

Input: 3+2

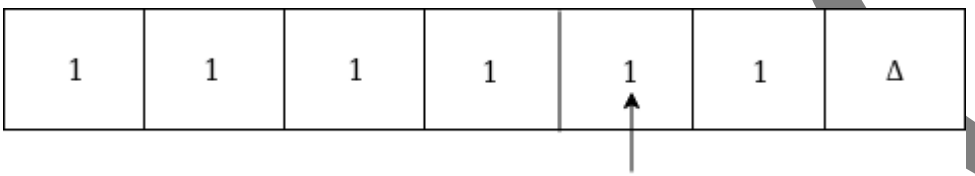
The simulation for 111+11Δ can be shown as below:



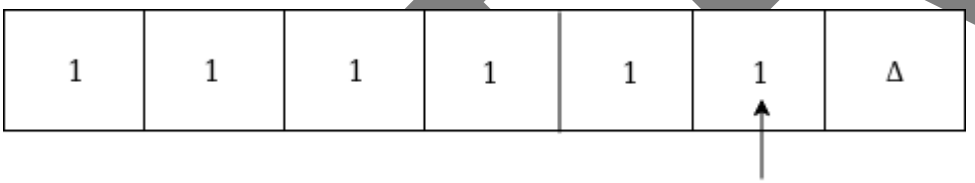
Move right up to + sign as:



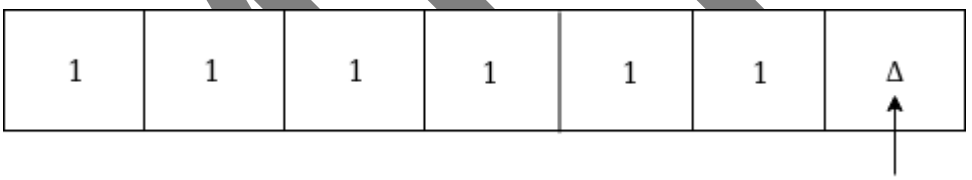
Convert + to 1 and move right as:



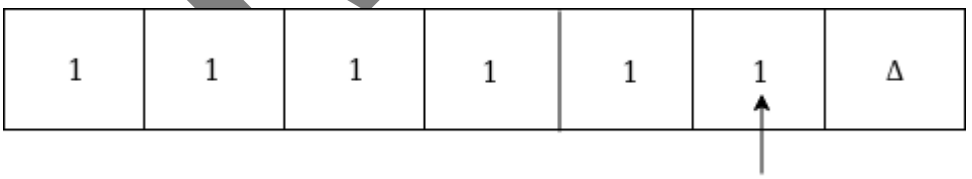
Now, move right



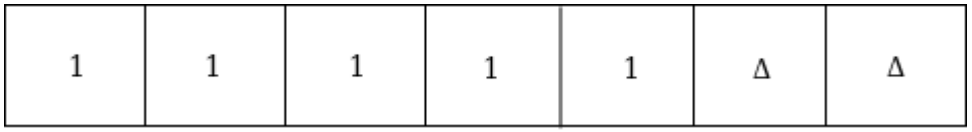
Again move right



Now Δ has encountered, so just move left as:



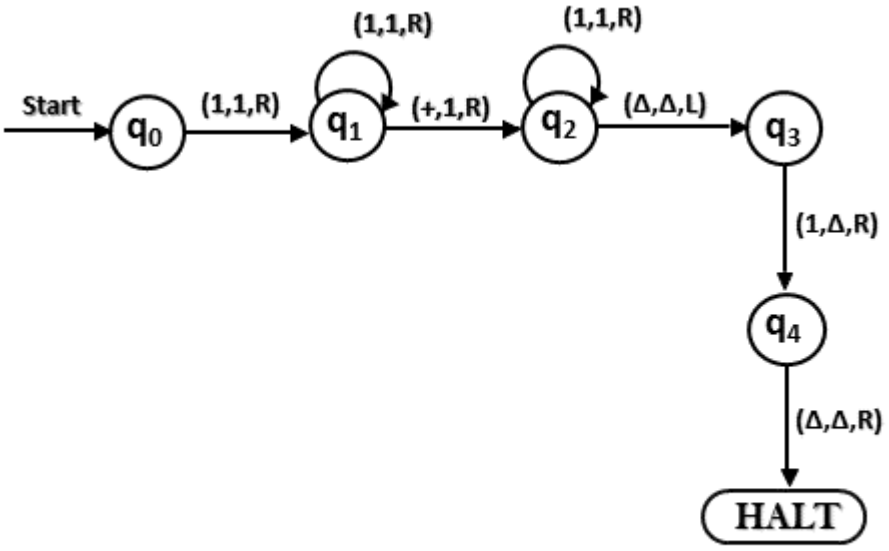
Convert 1 to Δ



Thus the tape now consists of the addition of two unary numbers.

The TM will look like as follows:

Here, we are implementing the function of f(a + b) = c. We assume a and b both are non zero elements.



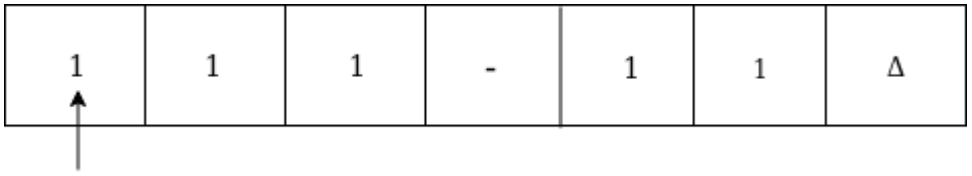
Example 5:

Construct a TM for subtraction of two unary numbers f(a–b) = c where a is always greater than b.

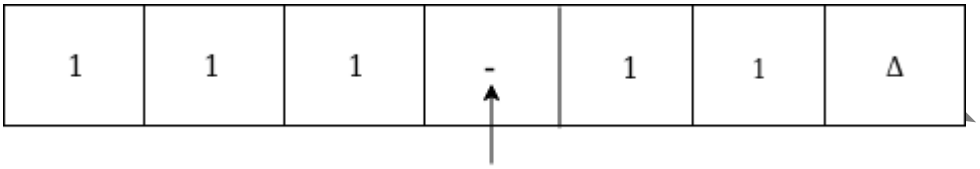
Solution: Here we have certain assumptions as the first number is greater than the second one. Let us assume that a = 3, b = 2, so the input tape will be:

1	1	1	-	1	1	Δ

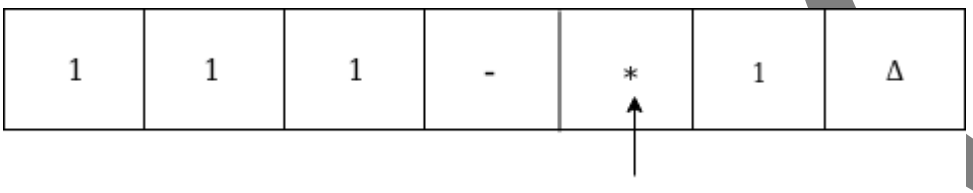
We will move right to - symbol as perform reduction of a number of 1's from the first number. Let us look at the simulation for understanding the logic:



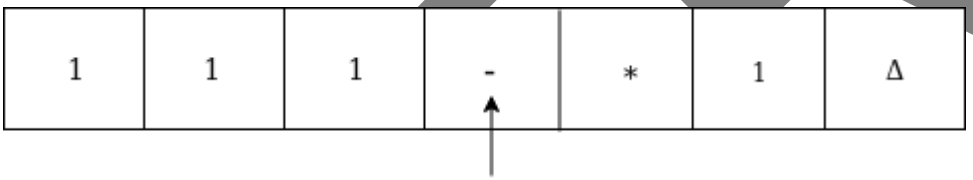
Move right up to - as:



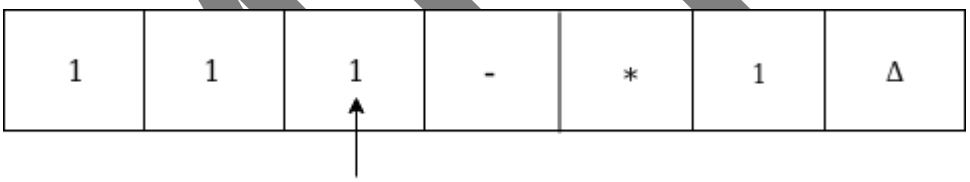
Move right and convert 1 to * as:



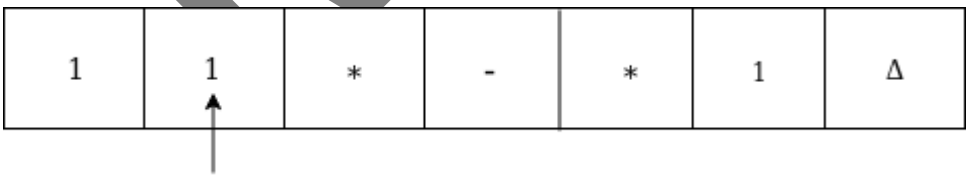
Now move left



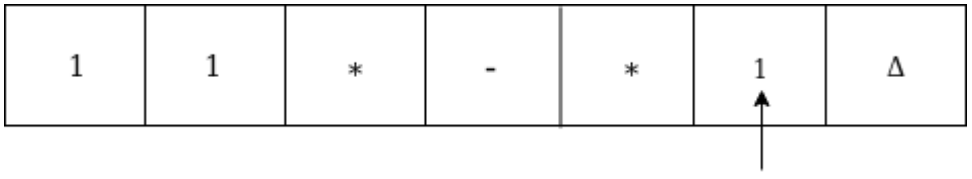
Again move left



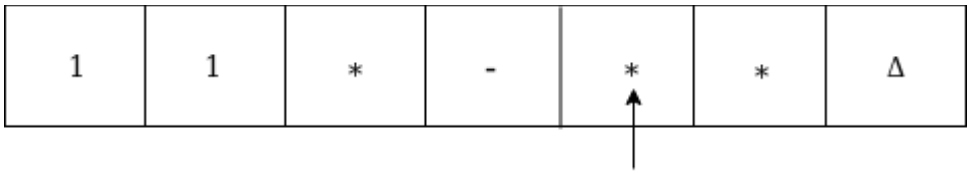
Convert 1 to * and move right-hand



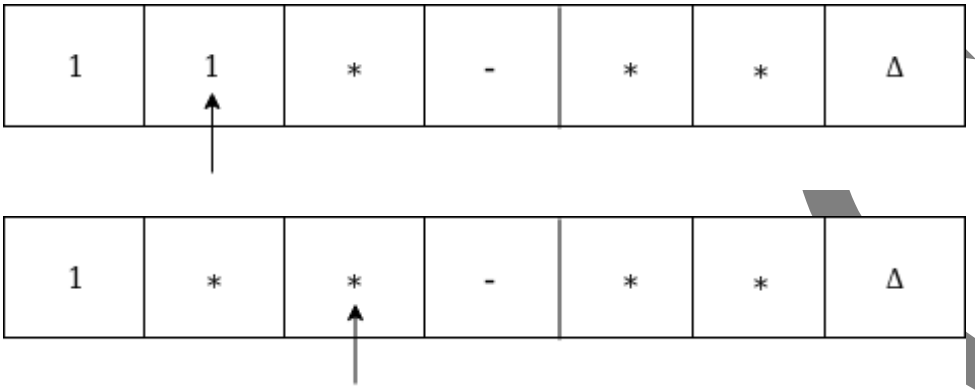
Now move right till 1



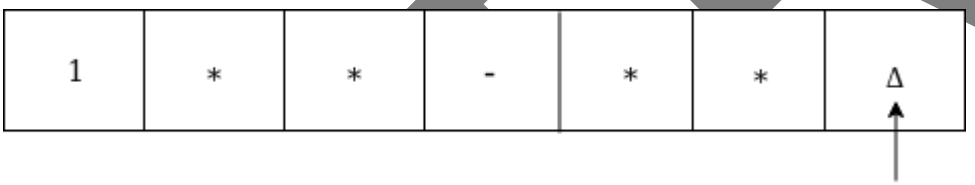
Convert 1 to * and move left



Convert 1 to * and move



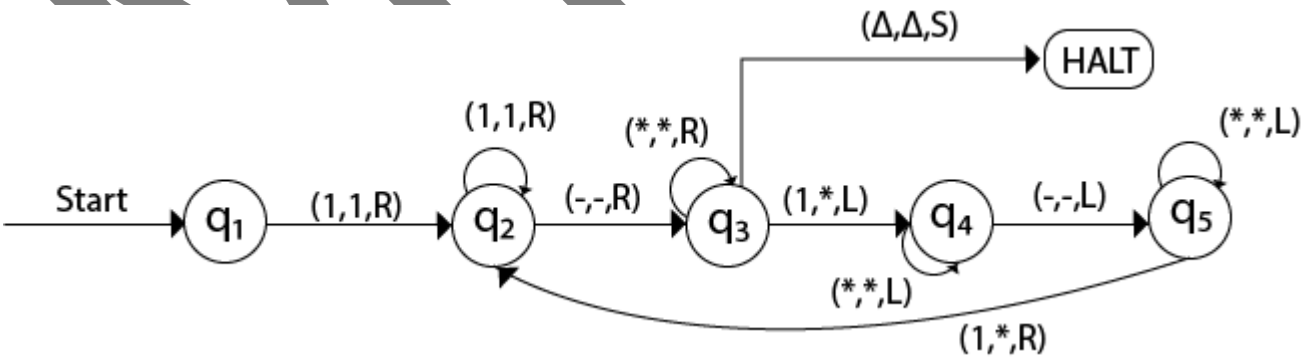
Move right till Δ as:



Now we are in the HALT state.

Thus we get 1 on the input tape as the answer for f(3-2).

The Turing machine will look like this:



Lecture-04

Universal Turing Machine

The Turing Machine (TM) is the machine level equivalent to a digital computer.

It was suggested by the mathematician Turing in the year 1930 and has become the most widely used model of computation in computability and complexity theory.

The model consists of an input and output. The input is given in binary format form on to the machine's tape and the output consists of the contents of the tape when the machine halts

The problem with the Turing machine is that a different machine must be constructed for every new computation to be performed for every input output relation.

This is the reason the Universal Turing machine was introduced which along with input on the tape takes the description of a machine M.

The Universal Turing machine can go on then to simulate M on the rest of the content of the input tape.

A Universal Turing machine can thus simulate any other machine.

The idea of connecting multiple Turing machine gave an idea to Turing –

- Can a Universal machine be created that can 'simulate' other machines?
- This machine is called as Universal Turing Machine

MITRC

This machine would have three bits of information for the machine it is simulating

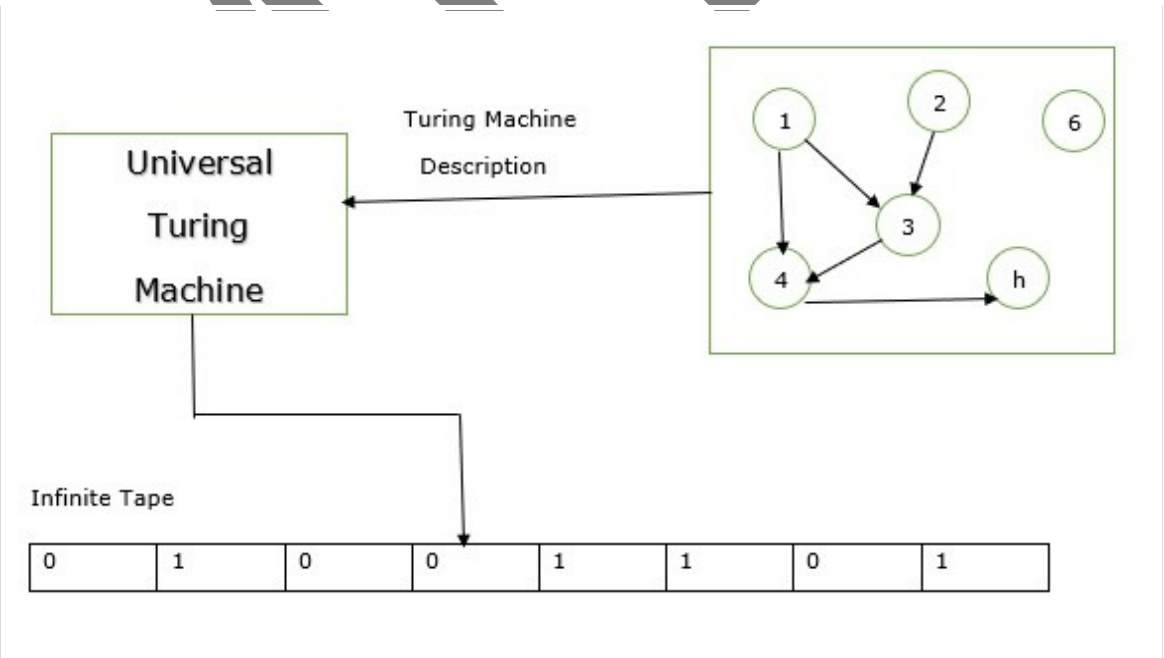
- A basic description of the machine.
- The contents of machine tape.
- The internal state of the machine.

The Universal machine would simulate the machine by looking at the input on the tape and the state of the machine.

It would control the machine by changing its state based on the input. This leads to the idea of a "computer running another computer".

It would control the machine by changing its state based on the input. This leads to the idea of a "computer running another computer".

The schematic diagram of the Universal Turing Machine is as follows –



Lecture-05

Multi-track Turing machines

Multi-track Turing machines, a specific type of Multi-tape Turing machine, contain multiple tracks but just one tape head reads and writes on all tracks. Here, a single tape head reads n symbols from n tracks at one step. It accepts recursively enumerable languages like a normal single-track single-tape Turing Machine accepts.

A Multi-track Turing machine can be formally described as a 6-tuple (Q, X, ∑, δ, q0, F) where −

- Q is a finite set of states
- X is the tape alphabet
- ∑ is the input alphabet
- δ is a relation on states and symbols where

  δ(Qi, [a1, a2, a3,....]) = (Qj, [b1, b2, b3,....], Left_shift or Right_shift)

- q0 is the initial state
- F is the set of final states

Note − For every single-track Turing Machine S, there is an equivalent multi-track Turing Machine M such that L(S) = L(M).

MITRC

Lecture-06

Properties of RL & REL Languages

Recursive Enumerable (RE) or Type -0 Language

RE languages or type-0 languages are generated by type-0 grammars. An RE language can be accepted or recognized by Turing machine which means it will enter into final state for the strings of language and may or may not enter into rejecting state for the strings which are not part of the language. It means TM can loop forever for the strings which are not a part of the language. RE languages are also called as Turing recognizable languages.

Recursive Language (REC)

A recursive language (subset of RE) can be decided by Turing machine which means it will enter into final state for the strings of language and rejecting state for the strings which are not part of the language. e.g.; L= {anbncn|n>=1} is recursive because we can construct a turing machine which will move to final state if the string is of the form anbncn else move to non-final state. So the TM will always halt in this case. REC languages are also called as Turing decidable languages. The relationship between

MITRC

Notes By: – Shashi Kant(Asst. Prof, CSE Department)

RE and REC languages can be shown in Figure 1.

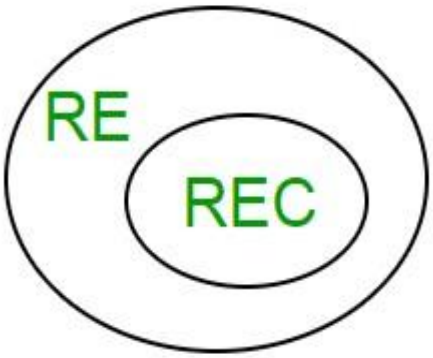


Figure 1

Closure Properties of Recursive Languages

- Union: If L1 and If L2 are two recursive languages, their union L1∪L2 will also be recursive because if TM halts for L1 and halts for L2, it will also halt for L1∪L2.
- Concatenation: If L1 and If L2 are two recursive languages, their concatenation L1.L2 will also be recursive. For Example:

  L1= {anbncn|n>=0}

  L2= {dmemfm|m>=0}

  L3= L1.L2

  = {anbncndm emfm|m>=0 and n>=0} is also recursive.

MIRC

L1 says n no. of a's followed by n no. of b's followed by n no. of c's. L2 says m no. of d's followed by m no. of e's followed by m no. of f's. Their concatenation first matches no. of a's, b's and c's and then matches no. of d's, e's and f's. So it can be decided by TM.

•Kleene Closure: If L1is recursive, its kleene closure L1* will also be recursive. For Example:

L1= {anbncn|n>=0}

L1*= { anbncn||n>=0}* is also recursive.

•Intersection and complement: If L1 and If L2 are two recursive languages, their intersection L1 ∩ L2 will also be recursive. For Example:

L1= {anbncndm|n>=0 and m>=0}

L2= {anbncndn|n>=0 and m>=0}

L3=L1 ∩ L2

= { anbncndn |n>=0} will be recursive.

L1 says n no. of a's followed by n no. of b's followed by n no. of c's and then any no. of d's. L2 says any no. of a's followed by n no. of b's followed by n no. of c's followed by n no. of d's. Their intersection says n no. of a's followed by n no. of b's followed by n no. of c's followed by n no. of d's. So it can be decided by turing machine, hence recursive. Similarly, complementof recursive language L1 which is ∑*-L1, will also be recursive.

Note: As opposed to REC languages, RE languages are not closed under complementon which means complement of RE language need not be RE.

MITRC

Lecture-07
CSL Languages AND Grammar

Context-Sensitive Grammar –

A Context-sensitive grammar is an Unrestricted grammar in which all the productions are of form –

$$\alpha \rightarrow \beta$$

where $\alpha, \beta \in (V \cup T)^+$ and $|\alpha| \leq |\beta|$

Where α and β are strings of non-terminals and terminals. Context-sensitive grammars are **more powerful** than context-free grammars because there are some languages that can be described by CSG but not by context-free grammars and CSL are less powerful than Unrestricted grammar. That's why context-sensitive grammars are positioned between context-free and

unrestricted grammars in the Chomsky hierarchy.



Context-sensitive grammar has 4-tuples. G = {N, Σ, P, S}, Where
N = Set of non-terminal symbols
Σ = Set of terminal symbols
S = Start symbol of the production
P = Finite set ofproductions All rules in P are of the form α1 A α2 –> α1 β α2

Context-sensitive Language: The language that can be defined by context-sensitive grammar is called CSL. Properties of CSL are :

- Union, intersection and concatenation of two context-sensitive languages is context-sensitive.
- Complement of a context-sensitive language is context-sensitive.

Example –

MIRC

Consider the following CSG.

S → abc/aAbc

Ab → bA

Ac → Bbcc

bB → Bb

aB → aa/aaA What is the language generated by this grammar?

Solution:

S → aAbc

→ a

bAc

→ abB

bcc

→ aBb

bcc

→ aaAb

bcc

→ aabA

bcc

→ aabb

Acc

→ aabbB

MITRC

bccc

→

aabBb

bccc

→

aaBbb

bccc

→

aaabb

bccc

The language generated by this grammar is {anbncn